



Міністерство освіти та науки України
Національний технічний університет України
«Київський політехнічний інститут імені Ігоря Сікорського»
Факультет інформатики та обчислювальної техніки
Кафедра інформатики і програмної інженерії

Звіт

з дисципліни «Технології розподілених обчислень»
Комп'ютерний практикум №6
«Розробка паралельного алгоритму множення матриць з
використанням MPI-методів обміну повідомленнями „один-до-
одного“ та дослідження його ефективності»

Виконав:

Студент III курсу
гр. ІІІ-33
Соколов О. В.

Прийняв:

Дифучин А. Ю.
“11” квітня 2026 р.

Київ – 2026

КОМП'ЮТЕРНИЙ ПРАКТИКУМ №5

Тема: Застосування високорівневих засобів паралельного програмування для побудови алгоритмів імітації та дослідження їх ефективності

Завдання:

1. Ознайомитись з методами блокуючого та неблокуючого обміну повідомленнями типу point-to-point (див. лекцію та документацію стандарту MPI).
2. Реалізувати алгоритм паралельного множення матриць з використанням розподілених обчислень в MPI з використанням методів блокуючого обміну повідомленнями (лістинг 1). 30 балів.
3. Реалізувати алгоритм паралельного множення матриць з використанням розподілених обчислень в MPI з використанням методів неблокуючого обміну повідомленнями. 30 балів.
4. Дослідити ефективність розподіленого обчислення алгоритму множення матриць при збільшенні розміру матриць та при збільшенні кількості вузлів, на яких здійснюється запуск програми. Порівняйте ефективність алгоритму при використанні блокуючих та неблокуючих методів обміну повідомленнями. 40 балів.

ВИКОНАННЯ

Завдання 1

Потрібно ознайомитись із методами блокуючого та неблокуючого обміну повідомленнями типу point-to-point згідно з лекцією та документацією стандарту MPI.

Блокуючий обмін. Виклики MPI_Send та MPI_Recv призупиняють виконання процесу до моменту, коли операцію можна безпечно вважати завершеною з точки зору семантики MPI: для MPI_Recv — дані повністю записані в буфер отримувача; для MPI_Send — буфер відправника можна знову використовувати (механізм буферизації залежить від реалізації). Це спрощує програмування, але може обмежувати перекриття обчислень і комунікації.

Неблокуючий обмін. Виклики MPI_Isend, MPI_Irecv ініціюють операцію й повертають MPI_Request. До MPI_Wait / MPI_Waitall не можна змінювати відповідні буфери. Після очікування гарантується завершення передачі та коректність даних у буфері прийому. Між постановкою операції та Wait можна виконувати інші дії, зокрема корисні обчислення. Комбінації типів надсилання й прийому дозволені, якщо дотримано узгодження тегів і джерел.

Додаткові поняття: комунікатор MPI_COMM_WORLD, ранг, тег, MPI_Status, вимір часу MPI_Wtime. Специфікації: mpi-forum.org/docs.

Таблиця 1.1 — Порівняльні риси блокуючого та неблокуючого point-to-point.

Аспект	Блокуючий (Send / Recv)	Неблокуючий (Isend / Irecv + Wait)
Повернення з виклику	Після завершення операції в сенсі MPI	Одразу після постановки в чергу
Буфер після виклику	Для Recv — дані готові; для Send — буфер вільний для повторного використання	До Wait — не змінювати відповідні буфери
Перекриття з обчисленнями	Обмежене	Можливе між I* та Wait
Складність для програміста	Нижча	Вища (порядок викликів, ризик deadlock)

Реалізовано демонстрацію на двох MPI-процесах: передача цілого числа від рангу 0 до рангу 1 спочатку блокуючими, далі неблокуючими викликами з MPI_Wait.

```
~/Documents/kpi/3_course/2_sem/parallel-computing-technologies/lab6/project git:(lab6) 7 files changed, 181 i
just task1

ld: warning: dylib (/opt/homebrew/Cellar/open-mpi/5.0.9/lib/libmpi.dylib) was built for newer macOS version (
15.0) than being linked (13.3)
=== Lab 6, task 1: point-to-point demo (2 processes) ===
[blocking] rank 0 sent value 42 to rank 1 (tag 100)
[blocking] rank 1 received 42 (source=0, tag=100)
[nonblocking] rank 0 Isend finished (sent 7)
=== Done ===
[nonblocking] rank 1 Irecv finished (received 7)
```

Run task2. ^ ↵

...l-computing-technologies/lab6/project lab6 20 • +956 -70

just task2 ↵

* ↵ new /agent conversation

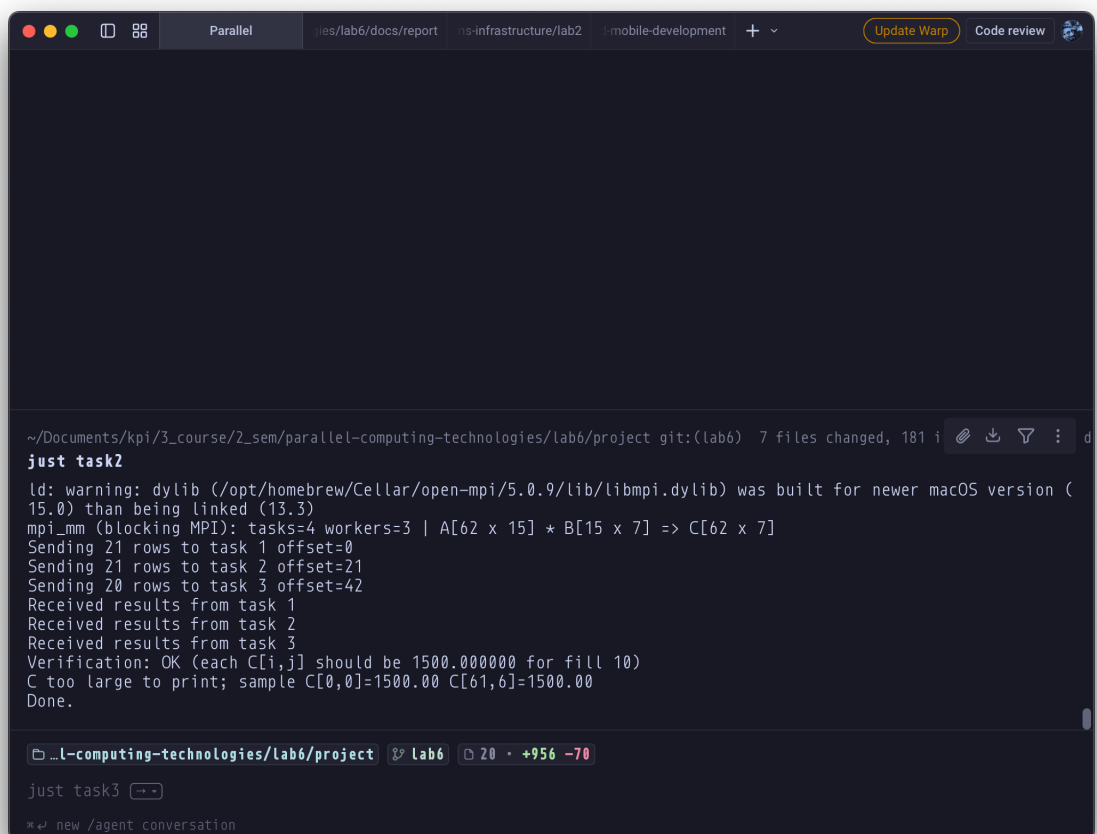
Рисунок 1.1 — Результат запуску демонстрації

Завдання 2

Потрібно реалізувати паралельне множення матриць $C = A \cdot B$ у MPI лише блокуючими викликами `MPI_Send` та `MPI_Recv`, за схемою з лістингу.

Алгоритм (схема master-workers). Процес з рангом 0 розподіляє смуги рядків матриці A між робочими процесами; кожному передається також повна матриця B . Робочі обчислюють відповідні фрагменти C і надсилають їх назад. Усі обміни виконуються блокуючими point-to-point викликами з узгодженими тегами для напрямків «від master» / «від worker».

Коректність реалізації перевірена на тестовому заповненні матриць (очікувані значення елементів C відомі аналітично).



```
~/Documents/kpi/3_course/2_sem/parallel-computing-technologies/lab6/project git:(lab6) 7 files changed, 181 i
just task2
ld: warning: dylib (/opt/homebrew/Cellar/open-mpi/5.0.9/lib/libmpi.dylib) was built for newer macOS version (
15.0) than being linked (13.3)
mpi_mm (blocking MPI): tasks=4 workers=3 | A[62 x 15] * B[15 x 7] => C[62 x 7]
Sending 21 rows to task 1 offset=0
Sending 21 rows to task 2 offset=21
Sending 20 rows to task 3 offset=42
Received results from task 1
Received results from task 2
Received results from task 3
Verification: OK (each C[i,j] should be 1500.000000 for fill 10)
C too large to print; sample C[0,0]=1500.00 C[61,6]=1500.00
Done.
```

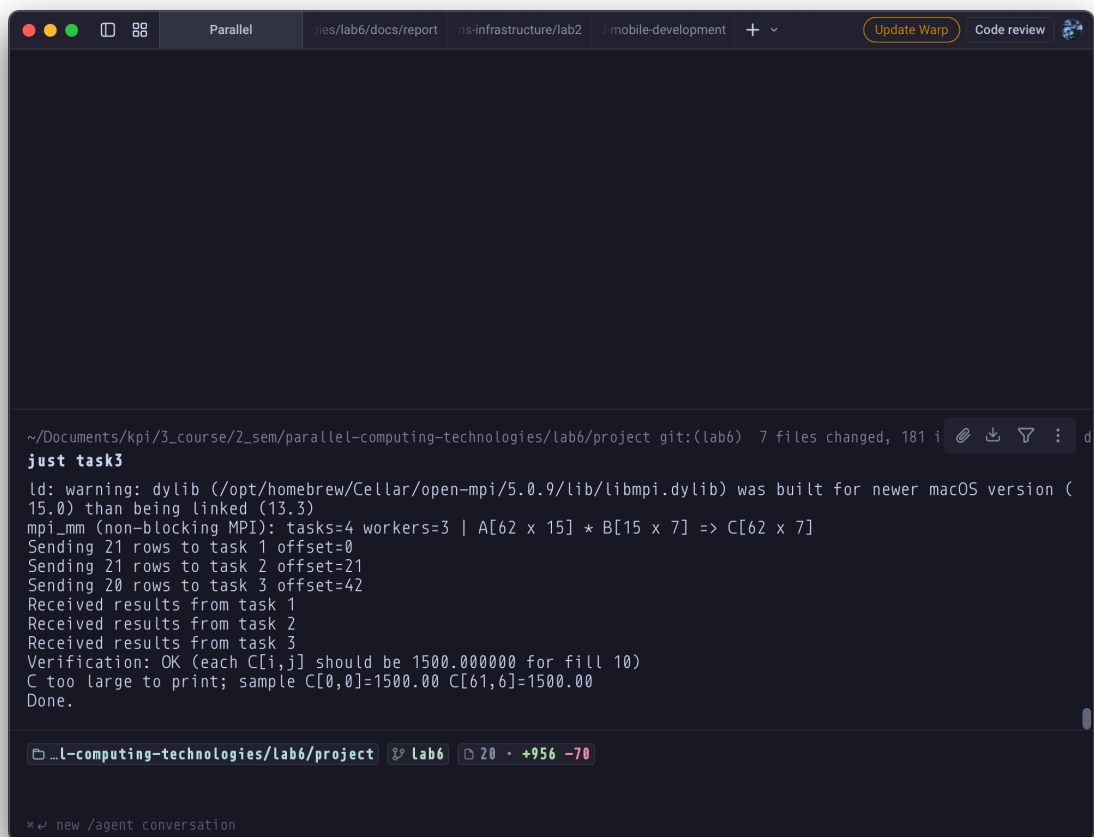
Рисунок 2.1 — Приклад виводу програми після успішного обчислення

Завдання 3

Потрібно реалізувати те саме множення $C = A \cdot B$ з неблокуючими викликами `MPI_Isend`, `MPI_Irecv` та `MPI_Wait` / `MPI_Waitall`.

Підхід. Топологія даних та роль master і workers та сама, що в завданні 2. Відмінність — у семантиці обміну: операції ініціюються неблокуючими викликами, а завершення явно синхронізується через `Wait`. Порядок постановки прийомів і очікувань має бути узгоджений із порядком надсилання, щоб уникнути взаємоблокувань і забезпечити коректність даних до початку обчислень на буферах.

Коректність підтверджена тими самими тестами, що й для блокуючого варіанту.



```
~/Documents/kpi/3_course/2_sem/parallel-computing-technologies/lab6/project git:(lab6) 7 files changed, 181 i
just task3
ld: warning: dylib (/opt/homebrew/Cellar/open-mpi/5.0.9/lib/libmpi.dylib) was built for newer macOS version (
15.0) than being linked (13.3)
mpi_mm (non-blocking MPI): tasks=4 workers=3 | A[62 x 15] * B[15 x 7] => C[62 x 7]
Sending 21 rows to task 1 offset=0
Sending 21 rows to task 2 offset=21
Sending 20 rows to task 3 offset=42
Received results from task 1
Received results from task 2
Received results from task 3
Verification: OK (each C[i,j] should be 1500.000000 for fill 10)
C too large to print; sample C[0,0]=1500.00 C[61,6]=1500.00
Done.
```

Рисунок 3.1 — Приклад виводу програми

Завдання 4

Потрібно дослідити ефективність розподіленого множення матриць при зміні розміру задачі та кількості MPI-процесів, а також порівняти блокуючий і неблокуючий варіанти обміну.

Фіксується час виконання вимірюваної фази на процесі master за допомогою `MPI_Wtime`: від початку розсилки даних робочим до завершення прийому всіх фрагментів результату C . У цей інтервал входять комунікація та паралельні обчислення на `workers`; підготовка вхідних даних і перевірка результату не включаються у вимір, щоб порівняння відображало саме «робочу» фазу алгоритму.

Розглядалися квадратні матриці розміру $N \times N$; множення $N \times N$ на $N \times N$. Параметри сітки експериментів (за замовчуванням): $N \in \{100, 150, \dots, 1000\}$ (крок 50); кількість процесів MPI (параметр запуску `-np`) від 2 до 8 inclusive. Для зменшення впливу випадкових флуктуацій кожену комбінацію (N, P) повторювали кілька разів (наприклад, 10 прогонів); для аналізу використовували найкращий (мінімальний) час серед прогонів. Окремо фіксувалися виміри для блокуючої та неблокуючої реалізацій.

За табличними даними побудовано теплові карти часу (у логарифмічній шкалі) для кожного варіанту обміну, теплову карту відношення часів блокуючого до неблокуючого варіанту (у логарифмічній шкалі відношення), а також лінійні зрізи: залежність часу від N при обраних P і залежність часу від P при обраних N .

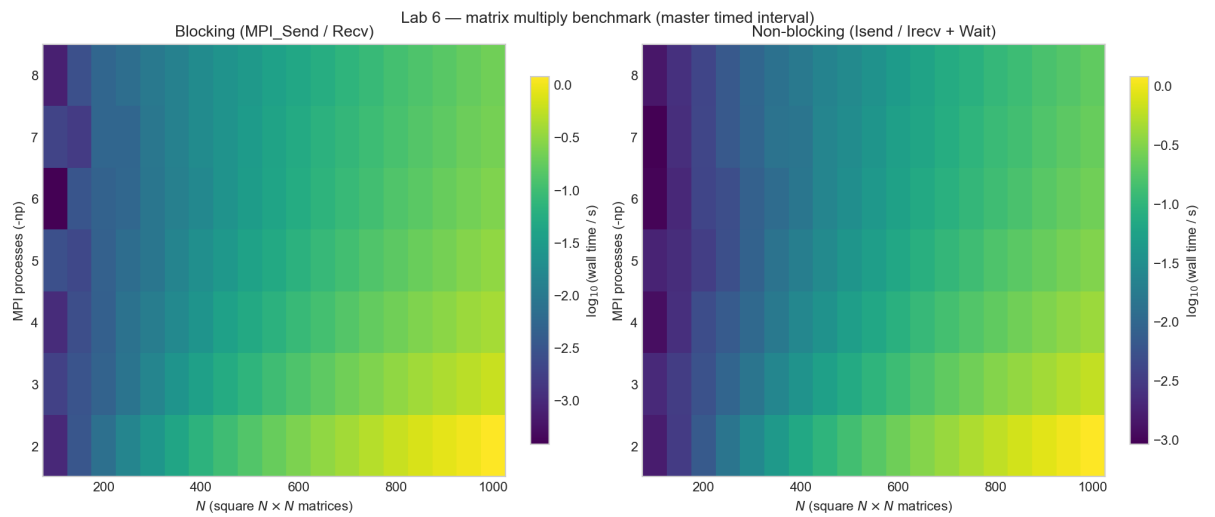


Рисунок 4.1 — Теплові карти $\log_{10}$ часу: блокуючий і неблокуючий обмін



Рисунок 4.2 — Відношення часів (блокуючий / неблокуючий), $\log_2$

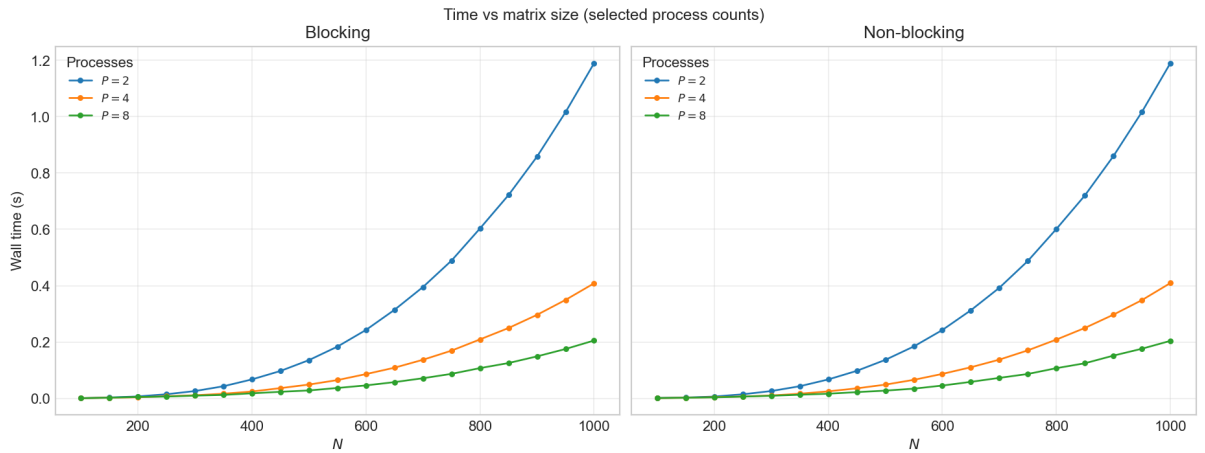


Рисунок 4.3 — Час від N при фіксованих P

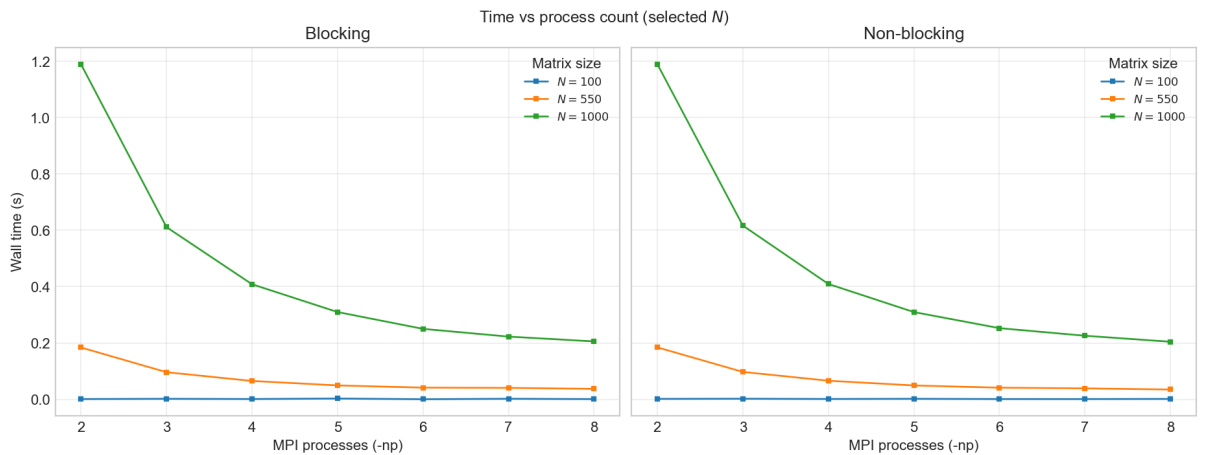


Рисунок 4.4 — Час від P при фіксованих N

([../results/lab6_slice_time_vs_procs.png](#)).

На рисунках 4.1–4.4 видно, як змінюється час при зростанні N та при збільшенні P , і наскільки близькі криві для двох варіантів обміну. Для одного обчислювального вузла та обраної схеми розподілу даних різниця між блокуючим і неблокуючим варіантами може бути невеликою, оскільки вузьке місце часто визначають обсяг передачі B до кожного worker та послідовна роль master, а не лише форма викликів MPI. Прискорення відносно базового P та ефективність $E = S/P$ доцільно обчислити за власними таблицями; для класичного матричного множення очікується ріст обчислювальної складності $\Theta(N^3)$ при фіксованому P .

Висновок

У межах практикуму №6 опрацьовано теоретичні основи MPI point-to-point, реалізовано паралельне множення матриць з блокуючим та неблокуючим обміном повідомленнями та виконано експериментальне дослідження ефективності залежно від розміру задачі й кількості процесів.

Код програми

Посилання

на

GitHub

репозиторій:

<https://github.com/sokolovgit/parallel-computing-technologies>

```
=== task1_p2p.c ===
/**
 * Lab 6, task 1: minimal point-to-point demo – blocking
 * (MPI_Send/MPI_Recv)
 * and non-blocking (MPI_Isend/MPI_Irecv + MPI_Wait).
 * Run: mpirun -np 2 ./bin/task1_p2p (or: just task1)
 */
#include <mpi.h>
#include <stdio.h>
#include <stdlib.h>

enum { TAG_BLOCK = 100, TAG_NONBLOCK = 200 };

int main(int argc, char **argv) {
    MPI_Init(&argc, &argv);

    int rank;
    int size;
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &size);

    if (size < 2) {
        if (rank == 0) {
            fprintf(stderr, "Need at least 2 MPI processes (e.g. mpirun -
np 2 ...).\n");
        }
        MPI_Finalize();
        return 1;
    }

    if (rank == 0) {
        printf("=== Lab 6, task 1: point-to-point demo (%d processes)
===\n", size);
        fflush(stdout);
    }

    /* Blocking exchange: rank 0 -> rank 1 */
    if (rank == 0) {
        int msg = 42;
        MPI_Send(&msg, 1, MPI_INT, 1, TAG_BLOCK, MPI_COMM_WORLD);
        printf("[blocking] rank 0 sent value %d to rank 1 (tag %d)\n",
msg, TAG_BLOCK);
        fflush(stdout);
    } else if (rank == 1) {
        int buf = 0;
        MPI_Status st;
        MPI_Recv(&buf, 1, MPI_INT, 0, TAG_BLOCK, MPI_COMM_WORLD, &st);
        printf("[blocking] rank 1 received %d (source=%d, tag=%d)\n",
buf,
st.MPI_SOURCE,
st.MPI_TAG);
        fflush(stdout);
    }
}
```

```

    }

    MPI_Barrier(MPI_COMM_WORLD);

    /* Non-blocking: rank 0 -> rank 1 */
    if (rank == 0) {
        int msg = 7;
        MPI_Request req;
        MPI_Isend(&msg, 1, MPI_INT, 1, TAG_NONBLOCK, MPI_COMM_WORLD,
&req);
        /* Buffer msg must not be touched until Wait completes. */
        MPI_Wait(&req, MPI_STATUS_IGNORE);
        printf("[nonblocking] rank 0 Isend finished (sent %d)\n", msg);
        fflush(stdout);
    } else if (rank == 1) {
        int buf = 0;
        MPI_Request req;
        MPI_Irecv(&buf, 1, MPI_INT, 0, TAG_NONBLOCK, MPI_COMM_WORLD,
&req);
        MPI_Wait(&req, MPI_STATUS_IGNORE);
        printf("[nonblocking] rank 1 Irecv finished (received %d)\n",
buf);
        fflush(stdout);
    }

    MPI_Barrier(MPI_COMM_WORLD);

    if (rank == 0) {
        printf("=== Done ===\n");
        fflush(stdout);
    }

    MPI_Finalize();
    return 0;
}

```

=== task2_mm_blocking.c ===

```

/**
 * Lab 6, task 2: parallel matrix multiply  $C = A * B$  using blocking
MPI_Send / MPI_Recv.
 * Master (rank 0) distributes row blocks of A and full B to workers;
workers return blocks of C.
 * Based on LLNL "mpi_mm.c" (Blaise Barney); fixed typos from the lab
handout; dynamic allocation.
 *
 * Usage: mpirun -np P ./task2_mm_blocking [NRA NCA NCB] [--bench]
 * Default dimensions: 62 x 15 and 15 x 7 (listing 1). Need P >= 2.
 * --bench: one CSV line to stdout; quiet (task 4 timing).
 */
#include <math.h>
#include <mpi.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#define MASTER 0
#define TAG_FROM_MASTER 1
#define TAG_FROM_WORKER 2

```

```

static void usage(const char *prog) {
    fprintf(stderr, "Usage: %s [NRA NCA NCB] [--bench]\n", prog);
    fprintf(stderr, "   Default: 62 15 7 (matrix A is NRA x NCA, B is NCA x NCB)\n");
    fprintf(stderr, "   --bench: quiet; print one CSV line to stdout (variant, dims, procs, wall seconds)\n");
}

int main(int argc, char **argv) {
    int nra = 62;
    int nca = 15;
    int ncb = 7;
    int bench = 0;
    int eff_argc = argc;
    if (eff_argc >= 2 && strcmp(argv[eff_argc - 1], "--bench") == 0) {
        bench = 1;
        eff_argc--;
    }

    if (eff_argc == 1) {
        /* defaults */
    } else if (eff_argc == 4) {
        nra = atoi(argv[1]);
        nca = atoi(argv[2]);
        ncb = atoi(argv[3]);
        if (nra < 1 || nca < 1 || ncb < 1) {
            usage(argv[0]);
            return 1;
        }
    } else {
        usage(argv[0]);
        return 1;
    }

    MPI_Init(&argc, &argv);

    int numtasks;
    int taskid;
    MPI_Comm_size(MPI_COMM_WORLD, &numtasks);
    MPI_Comm_rank(MPI_COMM_WORLD, &taskid);

    if (numtasks < 2) {
        if (taskid == MASTER) {
            fprintf(stderr, "Need at least two MPI processes.\n");
        }
        MPI_Abort(MPI_COMM_WORLD, 1);
    }

    int numworkers = numtasks - 1;

    if (taskid == MASTER) {
        size_t sa = (size_t)nra * (size_t)nca;
        size_t sb = (size_t)nca * (size_t)ncb;
        size_t sc = (size_t)nra * (size_t)ncb;
        double *a = malloc(sa * sizeof(double));
        double *b = malloc(sb * sizeof(double));
        double *c = calloc(sc, sizeof(double));
    }
}

```

```

    if (!a || !b || !c) {
        perror("malloc");
        MPI_Abort(MPI_COMM_WORLD, 1);
    }

    for (int i = 0; i < nra; i++) {
        for (int j = 0; j < nca; j++) {
            a[(size_t)i * (size_t)nca + (size_t)j] = 10.0;
        }
    }
    for (int i = 0; i < nca; i++) {
        for (int j = 0; j < ncb; j++) {
            b[(size_t)i * (size_t)ncb + (size_t)j] = 10.0;
        }
    }

    int averow = nra / numworkers;
    int extra = nra % numworkers;
    int offset = 0;

    if (!bench) {
        printf(
            "mpi_mm (blocking MPI): tasks=%d workers=%d | A[%d x %d] * B[%d x %d] => C[%d x %d]\n",
            numtasks,
            numworkers,
            nra,
            nca,
            nca,
            ncb,
            nra,
            ncb);
    }

    double t0 = MPI_Wtime();
    for (int dest = 1; dest <= numworkers; dest++) {
        int rows = (dest <= extra) ? averow + 1 : averow;
        if (!bench) {
            printf("Sending %d rows to task %d offset=%d\n", rows,
                dest, offset);
        }
        MPI_Send(&offset, 1, MPI_INT, dest, TAG_FROM_MASTER,
            MPI_COMM_WORLD);
        MPI_Send(&rows, 1, MPI_INT, dest, TAG_FROM_MASTER,
            MPI_COMM_WORLD);
        MPI_Send(
            a + (size_t)offset * (size_t)nca,
            rows * nca,
            MPI_DOUBLE,
            dest,
            TAG_FROM_MASTER,
            MPI_COMM_WORLD);
        MPI_Send(b, nca * ncb, MPI_DOUBLE, dest, TAG_FROM_MASTER,
            MPI_COMM_WORLD);
        offset += rows;
    }

    MPI_Status status;

```

```

        for (int source = 1; source <= numworkers; source++) {
            int off;
            int rows;
            MPI_Recv(&off, 1, MPI_INT, source, TAG_FROM_WORKER,
MPI_COMM_WORLD, &status);
            MPI_Recv(&rows, 1, MPI_INT, source, TAG_FROM_WORKER,
MPI_COMM_WORLD, &status);
            MPI_Recv(
                c + (size_t)off * (size_t)ncb,
                rows * ncb,
                MPI_DOUBLE,
                source,
                TAG_FROM_WORKER,
                MPI_COMM_WORLD,
                &status);
            if (!bench) {
                printf("Received results from task %d\n", source);
            }
        }
        double t1 = MPI_Wtime();
        if (bench) {
            printf("blocking,%d,%d,%d,%d,%.9f\n", nra, nca, ncb, numtasks,
t1 - t0);
        }

        double expected = (double)nca * 100.0;
        int ok = 1;
        for (int i = 0; i < nra && ok; i++) {
            for (int j = 0; j < ncb; j++) {
                double v = c[(size_t)i * (size_t)ncb + (size_t)j];
                if (fabs(v - expected) > 1e-6) {
                    printf("Mismatch C[%d,%d]=%.6f expected %.6f\n", i, j,
v, expected);
                    ok = 0;
                    break;
                }
            }
        }
        if (!bench) {
            printf("Verification: %s (each C[i,j] should be %.6f for fill
10)\n", ok ? "OK" : "FAIL", expected);
        } else if (!ok) {
            fprintf(stderr, "Verification FAIL (bench mode)\n");
        }

        if (!bench && nra * ncb <= 400) {
            printf("*****\nResult matrix C:\n");
            for (int i = 0; i < nra; i++) {
                for (int j = 0; j < ncb; j++) {
                    printf("%.2f ", c[(size_t)i * (size_t)ncb +
(size_t)j]);
                }
                printf("\n");
            }
            printf("*****\n");
        } else if (!bench) {
            printf(

```



```

        "C too large to print; sample C[0,0]=%.2f\n",
        C[%d,%d]=%.2f\n",
        c[0],
        nra - 1,
        ncb - 1,
        c[(size_t)(nra - 1) * (size_t)ncb + (size_t)(ncb -
1)]];
    }
    if (!bench) {
        printf("Done.\n");
    }

    free(a);
    free(b);
    free(c);
} else {
    (void)bench;
    int offset;
    int rows;
    MPI_Status status;

    MPI_Recv(&offset, 1, MPI_INT, MASTER, TAG_FROM_MASTER,
MPI_COMM_WORLD, &status);
    MPI_Recv(&rows, 1, MPI_INT, MASTER, TAG_FROM_MASTER,
MPI_COMM_WORLD, &status);

    double *a = malloc((size_t)rows * (size_t)nca * sizeof(double));
    double *b = malloc((size_t)nca * (size_t)ncb * sizeof(double));
    double *c = malloc((size_t)rows * (size_t)ncb * sizeof(double));
    if (!a || !b || !c) {
        perror("malloc");
        MPI_Abort(MPI_COMM_WORLD, 1);
    }

    MPI_Recv(a, rows * nca, MPI_DOUBLE, MASTER, TAG_FROM_MASTER,
MPI_COMM_WORLD, &status);
    MPI_Recv(b, nca * ncb, MPI_DOUBLE, MASTER, TAG_FROM_MASTER,
MPI_COMM_WORLD, &status);

    for (int k = 0; k < ncb; k++) {
        for (int i = 0; i < rows; i++) {
            c[(size_t)i * (size_t)ncb + (size_t)k] = 0.0;
            for (int j = 0; j < nca; j++) {
                c[(size_t)i * (size_t)ncb + (size_t)k] +=
                    a[(size_t)i * (size_t)nca + (size_t)j] *
b[(size_t)j * (size_t)ncb + (size_t)k];
            }
        }
    }

    MPI_Send(&offset, 1, MPI_INT, MASTER, TAG_FROM_WORKER,
MPI_COMM_WORLD);
    MPI_Send(&rows, 1, MPI_INT, MASTER, TAG_FROM_WORKER,
MPI_COMM_WORLD);
    MPI_Send(c, rows * ncb, MPI_DOUBLE, MASTER, TAG_FROM_WORKER,
MPI_COMM_WORLD);

    free(a);

```

```

        free(b);
        free(c);
    }

    MPI_Finalize();
    return 0;
}

=== task3_mm_nonblocking.c ===
/**
 * Lab 6, task 3: parallel matrix multiply  $C = A * B$  using non-blocking
 * MPI_Isend / MPI_Irecv + Wait / Waitall.
 * Same data flow as task 2 (master distributes row strips of A and full
 * B; workers return C strips).
 *
 * Usage: mpirun -np P ./task3_mm_nonblocking [NRA NCA NCB] [--bench]
 * Default dimensions: 62 x 15 and 15 x 7. Need P >= 2.
 * --bench: one CSV line to stdout; quiet (task 4 timing).
 */
#include <math.h>
#include <mpi.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#define MASTER 0
#define TAG_FROM_MASTER 1
#define TAG_FROM_WORKER 2

static void usage(const char *prog) {
    fprintf(stderr, "Usage: %s [NRA NCA NCB] [--bench]\n", prog);
    fprintf(stderr, "    Default: 62 15 7 (matrix A is NRA x NCA, B is NCA x\n");
    fprintf(stderr, "    NCB)\n");
    fprintf(stderr, "    --bench: quiet; print one CSV line to stdout\n");
    fprintf(stderr, "    (variant, dims, procs, wall seconds)\n");
}

int main(int argc, char **argv) {
    int nra = 62;
    int nca = 15;
    int ncb = 7;
    int bench = 0;
    int eff_argc = argc;
    if (eff_argc >= 2 && strcmp(argv[eff_argc - 1], "--bench") == 0) {
        bench = 1;
        eff_argc--;
    }

    if (eff_argc == 1) {
        /* defaults */
    } else if (eff_argc == 4) {
        nra = atoi(argv[1]);
        nca = atoi(argv[2]);
        ncb = atoi(argv[3]);
        if (nra < 1 || nca < 1 || ncb < 1) {
            usage(argv[0]);
            return 1;
        }
    }

```

```

    } else {
        usage(argv[0]);
        return 1;
    }

    MPI_Init(&argc, &argv);

    int numtasks;
    int taskid;
    MPI_Comm_size(MPI_COMM_WORLD, &numtasks);
    MPI_Comm_rank(MPI_COMM_WORLD, &taskid);

    if (numtasks < 2) {
        if (taskid == MASTER) {
            fprintf(stderr, "Need at least two MPI processes.\n");
        }
        MPI_Abort(MPI_COMM_WORLD, 1);
    }

    int numworkers = numtasks - 1;

    if (taskid == MASTER) {
        size_t sa = (size_t)nra * (size_t)nca;
        size_t sb = (size_t)nca * (size_t)ncb;
        size_t sc = (size_t)nra * (size_t)ncb;
        double *a = malloc(sa * sizeof(double));
        double *b = malloc(sb * sizeof(double));
        double *c = calloc(sc, sizeof(double));
        if (!a || !b || !c) {
            perror("malloc");
            MPI_Abort(MPI_COMM_WORLD, 1);
        }

        for (int i = 0; i < nra; i++) {
            for (int j = 0; j < nca; j++) {
                a[(size_t)i * (size_t)nca + (size_t)j] = 10.0;
            }
        }
        for (int i = 0; i < nca; i++) {
            for (int j = 0; j < ncb; j++) {
                b[(size_t)i * (size_t)ncb + (size_t)j] = 10.0;
            }
        }

        int averow = nra / numworkers;
        int extra = nra % numworkers;
        int offset = 0;

        if (!bench) {
            printf(
                "mpi_mm (non-blocking MPI): tasks=%d workers=%d | A[%d
x %d] * B[%d x %d] => C[%d x %d]\n",
                numtasks,
                numworkers,
                nra,
                nca,
                nca,
                ncb,

```

```

        nra,
        ncb);
    }

    double t0 = MPI_Wtime();
    for (int dest = 1; dest <= numworkers; dest++) {
        int rows = (dest <= extra) ? averow + 1 : averow;
        if (!bench) {
            printf("Sending %d rows to task %d offset=%d\n", rows,
dest, offset);
        }
        MPI_Request sreq[4];
        MPI_Isend(&offset, 1, MPI_INT, dest, TAG_FROM_MASTER,
MPI_COMM_WORLD, &sreq[0]);
        MPI_Isend(&rows, 1, MPI_INT, dest, TAG_FROM_MASTER,
MPI_COMM_WORLD, &sreq[1]);
        MPI_Isend(
            a + (size_t)offset * (size_t)nca,
            rows * nca,
            MPI_DOUBLE,
            dest,
            TAG_FROM_MASTER,
            MPI_COMM_WORLD,
            &sreq[2]);
        MPI_Isend(b, nca * ncb, MPI_DOUBLE, dest, TAG_FROM_MASTER,
MPI_COMM_WORLD, &sreq[3]);
        MPI_Waitall(4, sreq, MPI_STATUS_IGNORE);
        offset += rows;
    }

    int *off_buf = calloc((size_t)(numworkers + 1), sizeof(int));
    int *rows_buf = calloc((size_t)(numworkers + 1), sizeof(int));
    MPI_Request *meta_req = malloc(sizeof(MPI_Request) * (size_t)(2 *
numworkers));
    MPI_Request *c_req = malloc(sizeof(MPI_Request) *
(size_t)numworkers);
    if (!off_buf || !rows_buf || !meta_req || !c_req) {
        perror("malloc");
        MPI_Abort(MPI_COMM_WORLD, 1);
    }

    for (int s = 1; s <= numworkers; s++) {
        MPI_Irecv(
            &off_buf[s],
            1,
            MPI_INT,
            s,
            TAG_FROM_WORKER,
            MPI_COMM_WORLD,
            &meta_req[(size_t)(s - 1) * 2U]);
        MPI_Irecv(
            &rows_buf[s],
            1,
            MPI_INT,
            s,
            TAG_FROM_WORKER,
            MPI_COMM_WORLD,
            &meta_req[(size_t)(s - 1) * 2U + 1U]);
    }

```

```

    }
    MPI_Waitall(2 * numworkers, meta_req, MPI_STATUS_IGNORE);

    for (int s = 1; s <= numworkers; s++) {
        MPI_Irecv(
            c + (size_t)off_buf[s] * (size_t)ncb,
            rows_buf[s] * ncb,
            MPI_DOUBLE,
            s,
            TAG_FROM_WORKER,
            MPI_COMM_WORLD,
            &c_req[(size_t)(s - 1)]);
    }
    MPI_Waitall(numworkers, c_req, MPI_STATUS_IGNORE);

    double t1 = MPI_Wtime();
    if (bench) {
        printf("nonblocking,%d,%d,%d,%d,%.9f\n", nra, nca, ncb,
numtasks, t1 - t0);
    }

    for (int s = 1; s <= numworkers; s++) {
        if (!bench) {
            printf("Received results from task %d\n", s);
        }
    }

    free(meta_req);
    free(c_req);
    free(off_buf);
    free(rows_buf);

    double expected = (double)nca * 100.0;
    int ok = 1;
    for (int i = 0; i < nra && ok; i++) {
        for (int j = 0; j < ncb; j++) {
            double v = c[(size_t)i * (size_t)ncb + (size_t)j];
            if (fabs(v - expected) > 1e-6) {
                printf("Mismatch C[%d,%d]=%.6f expected %.6f\n", i, j,
v, expected);
                ok = 0;
                break;
            }
        }
    }

    if (!bench) {
        printf("Verification: %s (each C[i,j] should be %.6f for fill
10)\n", ok ? "OK" : "FAIL", expected);
    } else if (!ok) {
        fprintf(stderr, "Verification FAIL (bench mode)\n");
    }

    if (!bench && nra * ncb <= 400) {
        printf("*****\nResult matrix C:\n");
        for (int i = 0; i < nra; i++) {
            for (int j = 0; j < ncb; j++) {
                printf("%.2f ", c[(size_t)i * (size_t)ncb +
(size_t)j]);
            }
        }
    }

```

```

        }
        printf("\n");
    }
    printf("*****\n");
} else if (!bench) {
    printf(
        "C too large to print; sample C[0,0]=%.2f
C[%d,%d]=%.2f\n",
        c[0],
        nra - 1,
        ncb - 1,
        c[(size_t)(nra - 1) * (size_t)ncb + (size_t)(ncb -
1)]);
}
if (!bench) {
    printf("Done.\n");
}

free(a);
free(b);
free(c);
} else {
    (void)bench;
    int offset = 0;
    int rows = 0;
    MPI_Request rmeta[2];
    MPI_Irecv(&offset, 1, MPI_INT, MASTER, TAG_FROM_MASTER,
MPI_COMM_WORLD, &rmeta[0]);
    MPI_Irecv(&rows, 1, MPI_INT, MASTER, TAG_FROM_MASTER,
MPI_COMM_WORLD, &rmeta[1]);
    MPI_Waitall(2, rmeta, MPI_STATUS_IGNORE);

    double *a = malloc((size_t)rows * (size_t)nca * sizeof(double));
    double *b = malloc((size_t)nca * (size_t)ncb * sizeof(double));
    double *c = malloc((size_t)rows * (size_t)ncb * sizeof(double));
    if (!a || !b || !c) {
        perror("malloc");
        MPI_Abort(MPI_COMM_WORLD, 1);
    }

    MPI_Request rdata[2];
    MPI_Irecv(a, rows * nca, MPI_DOUBLE, MASTER, TAG_FROM_MASTER,
MPI_COMM_WORLD, &rdata[0]);
    MPI_Irecv(b, nca * ncb, MPI_DOUBLE, MASTER, TAG_FROM_MASTER,
MPI_COMM_WORLD, &rdata[1]);
    MPI_Waitall(2, rdata, MPI_STATUS_IGNORE);

    for (int k = 0; k < ncb; k++) {
        for (int i = 0; i < rows; i++) {
            c[(size_t)i * (size_t)ncb + (size_t)k] = 0.0;
            for (int j = 0; j < nca; j++) {
                c[(size_t)i * (size_t)ncb + (size_t)k] +=
                    a[(size_t)i * (size_t)nca + (size_t)j] *
b[(size_t)j * (size_t)ncb + (size_t)k];
            }
        }
    }
}

```

```

        /* Post metadata first and wait: master only Irecv's C after all
(offset,rows) pairs arrive. */
        MPI_Request sback[3];
        MPI_Isend(&offset, 1, MPI_INT, MASTER, TAG_FROM_WORKER,
MPI_COMM_WORLD, &sback[0]);
        MPI_Isend(&rows, 1, MPI_INT, MASTER, TAG_FROM_WORKER,
MPI_COMM_WORLD, &sback[1]);
        MPI_Waitall(2, sback, MPI_STATUS_IGNORE);
        MPI_Isend(c, rows * ncb, MPI_DOUBLE, MASTER, TAG_FROM_WORKER,
MPI_COMM_WORLD, &sback[0]);
        MPI_Wait(&sback[0], MPI_STATUS_IGNORE);

        free(a);
        free(b);
        free(c);
    }

    MPI_Finalize();
    return 0;
}

```